0125. Valid Palindrome

Easy | Category: Two Pointers | LeetCode: https://leetcode.com/problems/valid-p

Problem

- After lowercasing and dropping non-alphanumeric characters, does the string read the same forwards and backwards?
- Example: "A man, a plan, a canal: Panama" → true; "race a car" → false.

Intuition

- A palindrome mirrors around its centre, so compare the outermost valid characters and move inward.
- Skipping junk characters in place avoids building a cleaned copy.

Brute Force

- Build a filtered lowercase string, reverse it, compare: O(n) time but O(n) extra space.

Optimal Approach

- L = 0, r = n-1.
- Move l right while s[l] is not alphanumeric; move r left likewise.
- If lower(s[l]) != lower(s[r]) return false; else l++, r--.
- Return true when the pointers cross.

Worked Example

```
s = "race a car".

l=r(0) r=r(9)   r==r ok
l=a(1) r=a(8)   ok
l=c(2) r=c(7)   ok
l=e(3) r=' '(6) skip -> r=a(5)   e != a -> false
```

Java Solution

```java
public class ValidPalindrome {
    public boolean isPalindrome(String s) {
        int l = 0, r = s.length() - 1;
        while (l < r) {
            while (l < r && !Character.isLetterOrDigit(s.charAt(l))) l++;
            while (l < r && !Character.isLetterOrDigit(s.charAt(r))) r--;
            if (Character.toLowerCase(s.charAt(l)) != Character.toLowerCase(s.charAt(r))) return false;
            l++;
            r--;
        }
        return true;
    }
}
```

Complexity

- Time: O(n). Space: O(1).

Pitfalls

- Digits count as alphanumeric ("0P" is false).
- Inner skip loops must also check l < r.